

NAAN MUDHALVAN PROJECT DOCUMENTATION

EduGenie: Google Gemini Powered Learning Assistant

AI-Powered Educational Assistant using FastAPI and a Web Interface

Project Documentation

1. Project Overview

EduGenie is a lightweight AI-powered educational assistant designed to simplify learning through generative AI. The system allows students to ask questions, understand complex concepts in simple language, generate quizzes, summarize educational passages, and receive personalized learning recommendations.

2. Problem Statement

Students often need quick explanations, revision material, quizzes, and structured learning guidance. Traditional resources may require searching through large amounts of content. EduGenie provides these learning functions through one simple web interface connected to AI services.

3. Objectives

- Provide concise answers to academic questions.
- Explain difficult concepts in simple and readable language.
- Generate multiple-choice quizzes from a topic or passage.
- Summarize long educational content for quick revision.
- Provide beginner-to-advanced learning recommendations.
- Offer a lightweight web application that can run on modest hardware.

4. Proposed System

The proposed system uses a FastAPI backend with separate modules for explanation, question answering, quiz generation, summarization, and learning recommendations. A simple HTML/CSS interface collects the learner's input and displays the generated result. The source documentation specifies Gemini for Q&A, quiz generation, summarization, and learning paths, and LaMini-Flan-T5 for concept explanation.

5. Technology Stack

Technology	Purpose
Python 3.10+	Application development
FastAPI	Backend REST API
Uvicorn	ASGI server
HTML + CSS	Frontend interface
Jinja2	HTML templating
Google Gemini API	Generative AI functions
LaMini-Flan-T5-783M	Local concept explanation

6. System Architecture

EduGenie High-Level Architecture

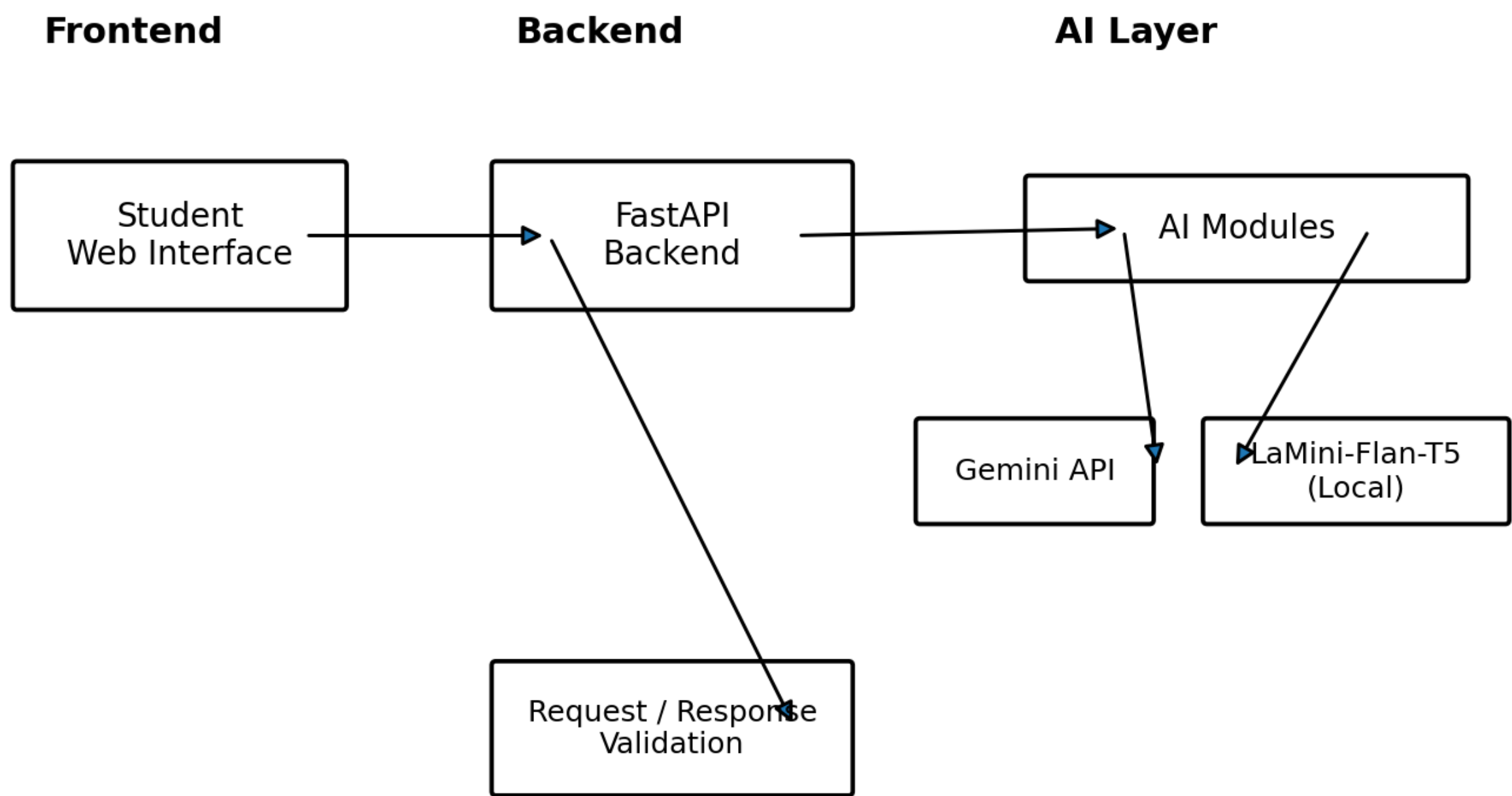


Figure 1. High-level architecture of EduGenie.

7. Folder Structure

The project documentation specifies the following core structure:

```
EduGenie/
├── main.py
├── explanation_module.py
├── qna.py
├── quiz_module.py
├── summary_module.py
├── learning_path.py
├── templates/
│   └── index.html
├── static/
│   └── style.css
└── requirements.txt
```

8. Functional Modules

8.1 Explanation Module

Uses the lightweight LaMini-Flan-T5 model to provide concise, context-aware explanations that break complex concepts into easier language.

8.2 Question and Answer Module

Uses Gemini to answer general knowledge and academic questions through the /qa endpoint.

8.3 Quiz Module

Generates three multiple-choice questions with four options each. The expected output is structured as JSON so it can be integrated into the application.

8.4 Summary Module

Summarizes long paragraphs into concise, easy-to-understand content while retaining the main information.

8.5 Learning Path Module

Generates a structured learning path from beginner to advanced levels and can include learning resources such as videos, articles, or books.

9. API Endpoints

Endpoint	Function
/qa	Question answering
/explain	Concept explanation
/quiz	Quiz generation
/summarize	Text summarization
/learn/recommendations	Personalized learning recommendations

10. System Flowchart

EduGenie System Flowchart

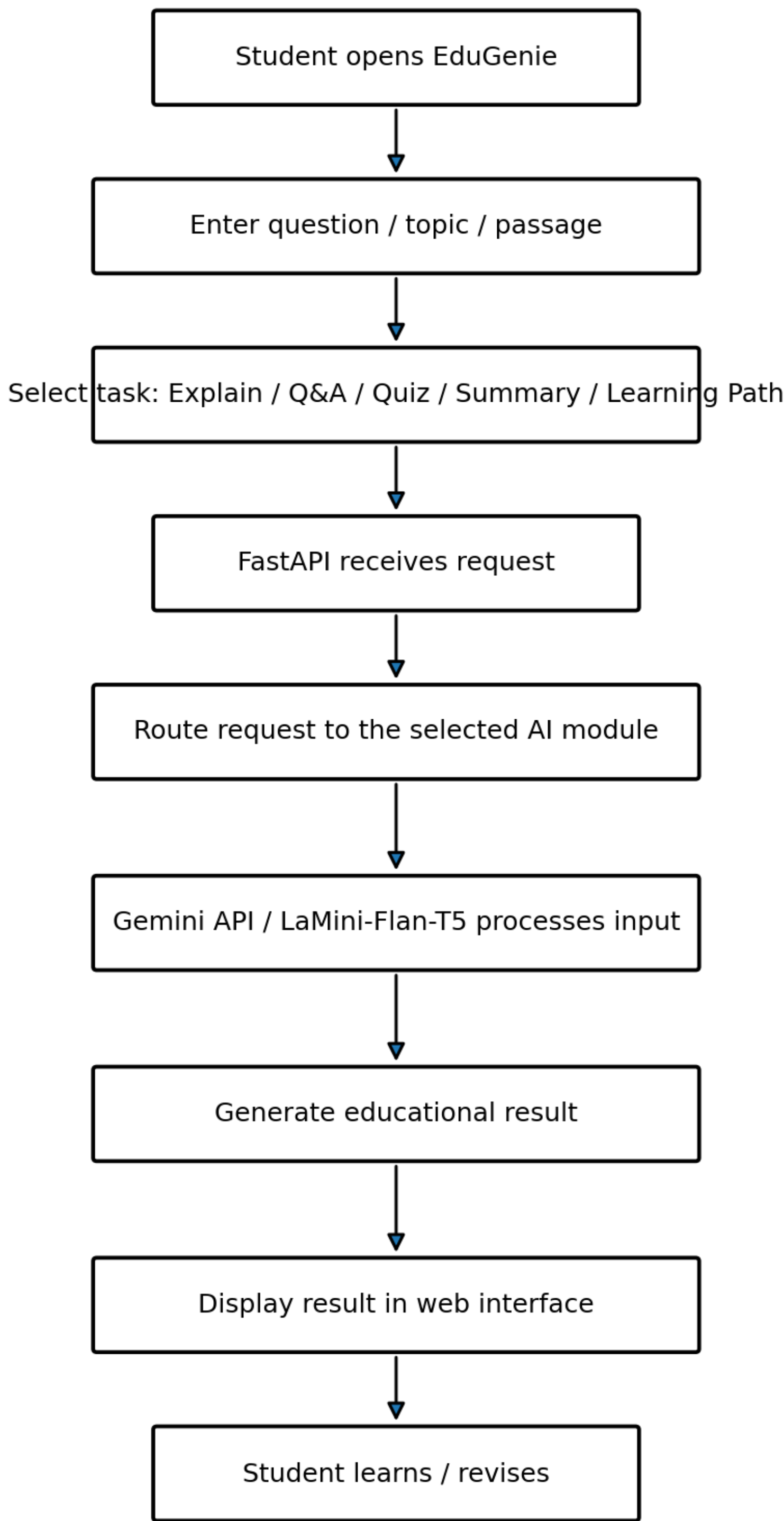
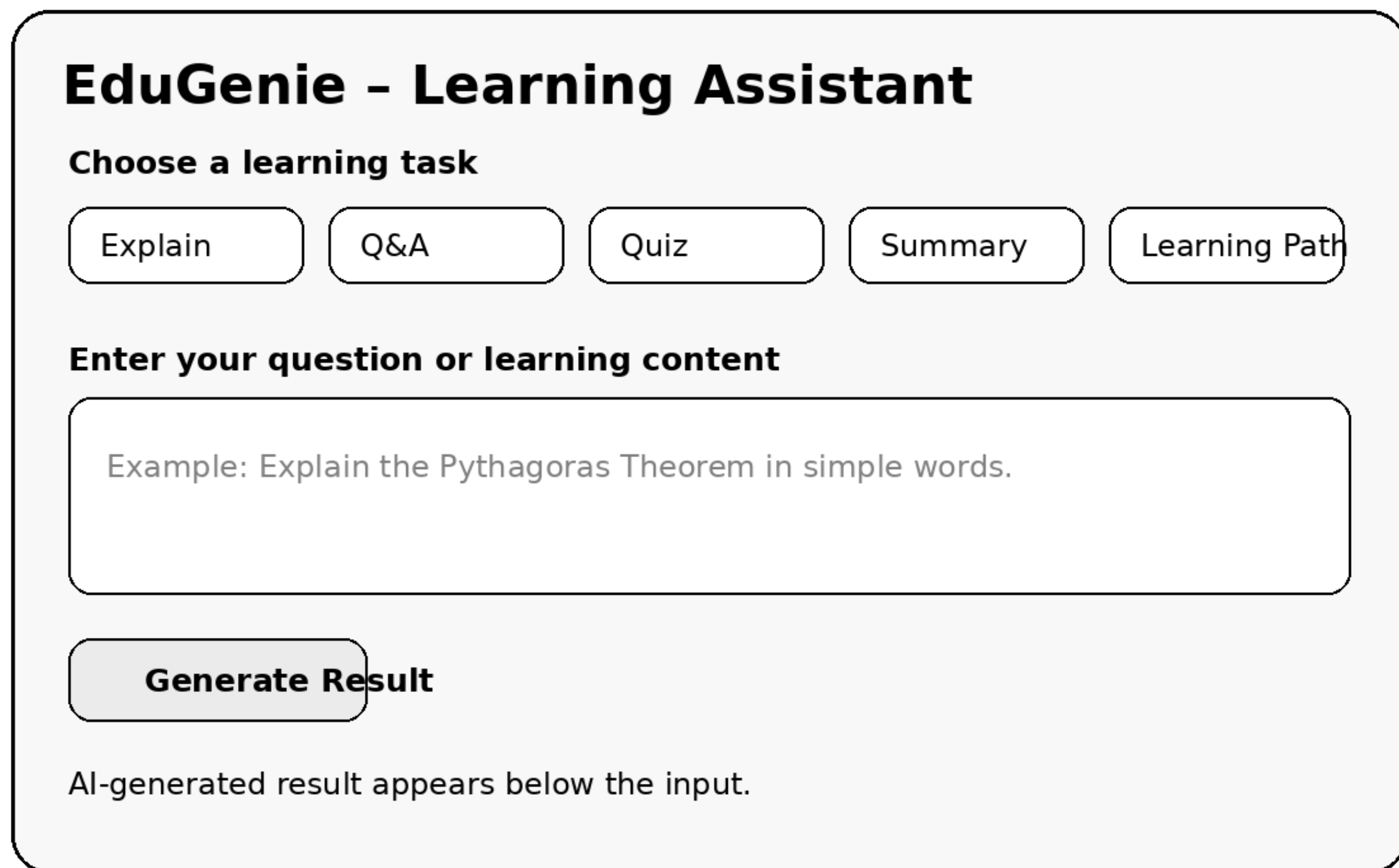


Figure 2. EduGenie end-to-end system flow.

11. Frontend Design

The frontend contains a task selection area, a text input area, a submit button, and a result section. The interface is designed to be responsive and simple for students.



The image shows a web interface for 'EduGenie - Learning Assistant'. It features a title, a task selection section with five buttons, a text input area with a placeholder example, a 'Generate Result' button, and a note about the AI-generated result.

EduGenie - Learning Assistant

Choose a learning task

Buttons: Explain, Q&A, Quiz, Summary, Learning Path

Enter your question or learning content

Example: Explain the Pythagoras Theorem in simple words.

Generate Result

AI-generated result appears below the input.

Figure 3. Illustrative EduGenie web interface.

12. Project Workflow / Milestones

- Milestone 1 – Model selection and architecture.
- Milestone 2 – Core functionality and FastAPI backend development.
- Milestone 3 – Web frontend development and live integration.
- Milestone 4 – Local deployment and functional testing.

13. Prerequisites and Installation

- Install Python 3.10 or newer and verify using `python --version`.
- Install the project dependencies from `requirements.txt`.
- Configure a Google Gemini API key securely.
- Install and use Uvicorn to run the FastAPI application.

The source documentation gives the local run command: `uvicorn main:app --reload`. After starting the server, open `http://127.0.0.1:8000` in a browser.

14. Functional Testing

Test Case	Input	Expected Result
TC01 – Q&A	Academic question	Concise answer is displayed
TC02 – Explanation	Complex topic	Simple explanation is displayed
TC03 – Quiz	Topic/passage	Three MCQs with four options are generated
TC04 – Summary	Long paragraph	Short summary is displayed
TC05 – Learning Path	Learning topic	Structured beginner-to-advanced path is displayed

15. Advantages

- Single interface for multiple learning tasks.
- Fast access to AI-generated educational assistance.
- Simple and lightweight web design.
- Modular backend makes individual features easier to maintain.
- Can support learners with different levels of prior knowledge.

16. Limitations

- AI-generated answers should be reviewed for accuracy.
- API availability and configuration affect cloud-based functions.
- Performance may vary depending on hardware and model usage.
- Internet access is required for cloud AI features described in the source project.

17. Future Enhancements

- Voice-based interaction.
- Multilingual support.
- Mobile application with offline capabilities.
- Learning progress dashboards.
- Gamification using badges, streaks, and challenges.
- Adaptive learning paths using learner data.
- Group study, teacher/parent dashboards, and LMS integration.
- Image and PDF input for doubt solving and summarization.
- Smart notifications and real-time synchronization.

18. Conclusion

EduGenie combines a simple web interface, FastAPI backend, and generative AI to create an accessible learning assistant. The project brings together question answering, concept explanation, quiz generation, summarization, and learning recommendations in one application. Its modular architecture also provides a foundation for future features such as voice interaction, multilingual learning, mobile access, progress tracking, and image/PDF-based learning.

19. Reference Project Information

This document is prepared specifically from the supplied EduGenie project documentation and is limited to the topic and functionality described there. The supplied source identifies the project as 'EduGenie: Google Gemini Powered Learning Assistant' and describes FastAPI, HTML/CSS, Gemini, LaMini-Flan-T5, the five functional endpoints, deployment, testing, conclusion, and future enhancements.

Submitted by

Name: Janani.A

Date:29.09.2026